

## Summer Research Project Logbook

### Week 1

Day 1 | 24/11/25 | 7.5 hrs

- <https://www.intel.com/content/www/us/en/support/programmable/support-resources/devices/arria-10-sx-support.html>
- Useful resources found for continuing the project such as the FGPA AI Suite and the OpenVINO Toolkit as I will ideally be switching to the Intel Aria 10 board instead of continuing with the cyclone.
- FPGA AI Suite already uses a built in IP for deploying AI inference onto the FPGA but since I already have designed my own custom architecture, I wont exactly be needing that. However, it does provide a more structured end-to-end pathway that may be of use as inspiration, especially on how to use the config files that can generated by the OpenVINO.
- OpenVINO (Open Visual Inference and Neural Network Optimisation) Toolkit is a software suite that is designed to accelerate AI inference across several platforms such as the FPGA that I am using.
  - I would be able to use NIOS II or could even migrate to NIOS V using the Heterogeneous Plugin to delegate my preprocessing to that coprocessor or just make NIOS host the OpenVino Runtime which would essentially become my overall Control Unit, which would manage the memory transactions and instructions for my NPU architecture that I have designed.
  - Would most likely use C++ however, the framework will need to be re-evaluated since AlexNet was used previously on Python script.

Potential Revamps based upon upgrade to Aria 10:

- Number of DSP blocks will be upgraded from 81 to ~1600
- M10K on chip memory to M20K blocks which would an increase from ~6mb to ~48mb

To proceed with the idea of using OpenVINO I would most likely have to use my arch linux OS on my computer to make it work with a C++ based framework instead of the AlexNet that I had on python. For that I would also need to install Intel Quartus Prime Pro

I have found a paper that discusses the analysis of the acceleration of deep learning inference models on a heterogenous architecture based on OpenVINO. This could help justify the use of the OpenVINO software and potentially help in using it.

<https://ieeexplore.ieee.org/document/9668607/citations#citations>

The paper discusses the use of the intel arria 10 FPGA alongside CPU or CNN inferencing compared to just a standard CPU inference, as that is extremely helpful when operating at the edge. Nothing that I already don't know. The most important part is the OpenVINO framework. It comprises of two parts: the intermediate representation (IR) which is the generated via Model Optimiser, converting pretrained model into .bin and .xml. .bin is binary file consisting of weights and biases, and .xml shows the structure of the layers and connectivity. The second part is the inference engine that consists of the C++ or python library which would provide the API for reading the intermediate representation.

## Day 2 | 25/11/25 | 5 Hrs

- Spent most of the time trying to fix errors to download the OpenVINO software on linux. Multiple errors during installation of C++ and python libraries and other version conflicts on my linux system.

Will continue on the research aspect of things to prepare further for when I get access to the Arria 10 FPGA board.

There is a hardware acceleration paper that discusses optimisation techniques when accelerating convolution operations on an Arria 10 FPGA board:

<https://ieeexplore.ieee.org/document/8330049>

### 1. Loop Unrolling (Parallelism)

- The main purpose of this is to maximise the parallelism of computations by mapping multiple loop iterations directly into dedicated hardware. This optimisation is crucial because it sets the factor by which computational speed is increased. How?
  - It designs the PE grid to perform MAC operations simultaneously per clock cycle
  - The degree of unrolling is defined by the set of design variables denoted as  $P^*$
  - The total number of parallel MAC operations ( $P_m$ ) is the product of these variables, which directly determines maximum speed ( $\text{Throughput} = N_m / P_m$ ).
  - Unrolling specific loops is necessary to exploit data reuse, which is vital for minimising memory access.
    - Unrolling Loop-3 ( $P_{ix} \times P_{iy}$ ) is the method used to achieve spatial reuse of weights, where one weight.
    - Unrolling Loop-4 ( $P_{of}$ ) is key to realising spatial reuse of input pixels/features.

### 2. Loop Tiling (Memory Partitioning)

- Loop tiling is necessary because the on-chip memory of an FPGA is insufficient to store the large data volumes associated with deep CNNs, requiring gigabytes of external memory (SDRAM).
  - Tiling divides the model's entire data set (pixels, weights, and intermediate results) into multiple smaller blocks that are sized specifically to fit within the limited capacity of the on-chip buffers.
  - The extent of tiling is governed by the  $T^*$  variables. These variables define the exact dimensions of the chunk of data loaded from external memory into the on-chip buffers.
  - The tiling variables determine the lower bound on the required size of the on-chip buffers.
  - Proper tiling is the primary mechanism for reducing the number of costly DRAM accesses, which in turn minimises latency and power consumption.

### 3. Loop interchange (Latency Factor)

- Loop interchange determines the flow and execution sequence of the convolution loops and is critical for managing transient data storage.
  - Defines the sequential order in which the four convolution loops are computed. This decision dictates the pattern of data movement both within the on-chip buffer to the PEs and from the external memory to the on-chip memory.
  - The primary objective of loop interchange is to minimise the number and movement of partial sums. A partial sum is the intermediate result of a MAC operation that must be stored until the final accumulation is complete.

- This optimisation is not very necessary to me because my existing design is an output stationary orthogonal architecture of Pes so I don't exactly have any partial sums to deal with.

### Day 3 | 26/11/25 | 5

Currently a bit confused about the DMA and AXI pipeline observed in the intel DMA reference design documentation. The Arria 10 supports PCIe Gen3 x8 links which is up to 8Gbps rate of data transfer and uses the Avalon-MM interface of DMA within the FPGA fabric. So this means that my NPU that I have designed must communicate through AXI4 or Avalon-MM I am currently unsure. Will have to confirm once I get the board and use the platform designer.

Upon research of the RISC-V acceleration, the sparsity calculations can be accelerated by creating a single custom instruction that the NIOS V executes, replacing multiple software instructions.

<https://arxiv.org/pdf/2511.06955>

This paper presents the acceleration of deep learning on resource-constrained edge FPGAs by fusing a soft-core processor with custom hardware acceleration. The complexity of this paper lies in its quantitative analysis of bottlenecks, which is crucial for maximising the efficiency of my Arria 10 design.

The central goal of the paper to solve the trilemma of Edge AI: balancing latency, power, and flexibility. Unlike previous works that treated hardware acceleration as a slow, generic memory-mapped coprocessor, this research uses ISA extensions to create a true hardware-software co-design.

The primary innovation is integrating the accelerator at the instruction level. This means the software running on the RISC-V core does not manage the accelerator by writing multiple values to random memory addresses. Instead the CPU issues a single custom assembly instruction (e.g. FPGA.VCONV) that directly triggers the entire complex hardware operation.

The architecture itself is a self-contained embedded system built around a RISC-V core deployed on the Xilinx Zynq-7020 SoC which contains an ARM Cortex-A9 as a baseline for comparison.

| Extension | Functionality & Hardware                                                                                                                                                                          | Perofmrnce Interpretation                                                                                                                                        |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FPGA.CONV | Vectorised Convolution: The core NPU accelerator. It is a 4x4 systolic array implemented with 32 DSP slices. The instruction repalcecs the entire loop nest of the convolution kernel in software | Achieved a 7.20x speedup over the highly optimisd ARM NEON baseline. This single instruction accounts for 60% to 75% of the total time saved in the application. |
| FPGA.GEMM | General Matrix Multiplication: An 8x8 systolic array optimised for tasks like fully connected layers                                                                                              | Achieved 4.20x speedup. The 8x8 array was found to be the optimal size for maximising efficiency without saturating the memory bandwidth.                        |
| FPGA.RELU | Vectorised Activation: Handles                                                                                                                                                                    | The speedup (3x) is less                                                                                                                                         |

|             |                                                                                                                                  |                                                                                                                                                 |
|-------------|----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
|             | non-linear activation functions (ReLU, LeakyReLU).                                                                               | dramatic but the instruction achieves an 85% reduction in instruction count. This frees up the CPU from managing repetitive scalar logic.       |
| FPGA.CUSTOM | Exensibility: An “escape hatch” providing a generic interface for specialised tasks (e.g. Non-Maximum Supression (NMS) for YOLO) | Demonstrates a critical principle: you don’t need a single monolithic NPU; specialised hardware blocks can be called via separate instructions. |

## Memory and Bottleneck Analysis (The Critical Trade-Offs)

The paper’s deep dive into hardware metrics reveals the true limitations of acceleration, which may help in my case but it is much easier since the Arria 10 has a lot more to off in terms of memory bandwidth. The overall latency speedup was 2.14x. This measured speedup achieved only 63% of the 3.39% theoretical maximum dictated by Amdahl’s Law.

The source of inefficiency was that 15% DMA Overhead and 12% Memory Bandwidth stalls. This meant that majority of the performance ceiling is due to data transfer delays, not slow computation. This mandates solutions like DMA batching and wider data paths.

The design used 64KB of BRAM on-chip memory integrated with the AXI interconnect. Triple-buffering was found to be essential to achieve peak performance, while double-buffering caused 18% performance loss due to stalls.

The AXI4 Data Bus measured at 850 MB/s. The core challenge for this is making the accelerator ask for data efficiently. The solution is intelligent tiling and AXI burst transfers to eliminate the overhead of initiating transactions for every small data piece.

## Day 4 | 27/11/25 | 5

### Meeting 1:

- Check OpenVino version, because latest might not work with FPGA
- Find ll license file, allocate the license file in bash
- something to do with WSL
- Try take advantage of the time, try to extend research skills, and possibly publish. Check out Modelzoo, so that OpenVino has access to several models not just AlexNet. It could also be used for LLMs not just CNNs.
- Try to pick out the research needs, and not explore too much unless you don’t know what you’re doing.
- Had few hours of fixing required for license errors for Quartus, I believe it is working now. The task for the rest of the day is to familiarise with the board, and possibly AI suite, OpenVINO and obviously Quartus pro.f

Plan for next week:

Upgrade the current set of Pes because currently its a 8x8 signed multiplier with a 32-bit accumulator and that is too little for maximising the use on the arria 10 board. Will aim for a 16-bit fixed point precision.

Upgrade memory system from basic ROMs to exploring the Avalon-MM/AXI4 interfaces which will facilitate later integration with the NIOS processor and DMA.

Main Questions that I need to answer:

- What exactly be the roles for each component on the software stack side of things?
- What difference between MM and AXI4?
- 

Day 5 | 01/12/25 | 4 Hours

Upgrading hardware logic according to the board. Until I faced the issue discussed in Day 6

Day 6 | 02/12/25 | 5 Hours

The large-scale 32x32 architecture immediately revealed logical shortcomings in the timing signals (ready, count) that the original 8x8 prototype did not show.

This initial problem was exposed by testing a 3x3 matrix multiplication where the latency is 7 cycles. There was a serious synchronisation and counter decoupling bug in the control unit. Basically the counter only ran for  $K=3$  cycles before the external ready signal pulsed low, prematurely stopping the count. So we removed the dependency on the external ready signal for counting and introduced a self-managing internal state (run\_enable). The counter now runs until it hits the calculated maximum latency ( $M+N+K-2$ ), independent of the external pulse duration. The input data gates were updated to rely on the internal run\_enable flag. This ensured the data streamed for the full required  $K$  cycles (32 cycles in the final test) before being safely replaced by padded zeros.

Once the core logic was fixed, the 32x32 test compiled and ran for the correct 94 cycles, however the output matrix was corrupted as it was only correct up until the 8<sup>th</sup> row and 20<sup>th</sup> column. This was discovered to be the testbench inputs being incorrect after the 8<sup>th</sup> row, so that was replaced. And now the final output is correct. Next step is to check if sparsity can be handled.

Day 7 | 04/12/2025 | 5 Hours

Row/col stripping sparsity handling does in fact work now. This has been tested with a modified testbench and recycling some old python code from p4p.

The next step is to architect a new top-level system in order to replace the previous ROM and FSM structure with a proper operational high-bandwidth memory controller and possibly some considerations with the FGPA to CPU interface suitable for my OpenVINO integration.

The current 'npu\_wrapper' and its components rely on low-bandwidth, synchronous ROM access and an FSM that loads the whole 32x32 matrix once. This is a simulation bottleneck that must be replaced by AXI/Avalon based memory hierarchy. In order to do so, a double buffer system using M20K BRAM blocks can be used to allow the accelerator to compute the current tile from Buffer A while the DMA loads the next tile into Buffer B.

Day 8 | 05/12/25 | 7 Hours

Day 9 | 08/12/25 | 7 Hours

Configured the on-chip memory to replace simulation constants in a way that maximum data throughput can be achieved. Implemented a 5 buffer system using M20K Trual Dual Port RAM blocks in a ping pong manner to maximise the locality and eliminate any NPU idle time. The two buffers for each input allow the DMA engine to write the next data tile into one buffer while the NPU reads the current tile from the other. This ensures the NPU is never stalled waiting for slow memory accesss, maintaining continuous DSP utilisation.

The buffers were designed as such:

- Two read/write ports with 16 bit width for the input buffers and 64 bit width for the output buffer. Single clock mode which just uses the CLOCK\_50 and the initial content is always stored on Port A for convenience and consistency. To ensure the system is stable upon reset and compatible with future Dynamic Partial Reconfiguration (DPR) goals, Output ACRL is selected for clearing output registers on asynchronous reset, ensuring a known zero state. Clock Enables are selected for low power to stop clock toggling. For the actual DPR there was a clock compatability selection which I just ticked.

For the M, N, K values, currently they will be hardcoded from a hardware wrapper but they will eventually be AXI registerss and will be read from those registers written a software coprocessor or even the HPS.

Is this the same as making a staging and local buffer?? – check out paper: [Bandwidth Optimisation Through On-chip memroy Restructuring for HLS](#)

Day 10 | 09/12/2025 | 7 Hours

Trying to make QuestaSim work

Day 11 | 10/12/2025 | 7 Hours

Same thing

Day 12 | 11/12/2025 | 8 Hours

Same old

Day 13 12/12/2025 | 4.5

Same issue ugghhhh

Day 14 | 14/12/25